CyberSNova Cyber Security Internship

Intermediate Project 2: PortSwigger Web Security Academy Labs

1. Intern Details

Name: Shaurya Gupta Email: shauryagupta0106@gmail.com Reference Number: CSN-OC-OL-26194 Registration Number: CSN-OC-2026-194 Domain: Cyber Security Project Level: Intermediate Project Number: 2 Month/Year: October 2026

2. Project Description

This project involves completing five Apprentice-level labs on PortSwigger Web Security Academy. The exercises investigate web application security weaknesses, demonstrate their impact in the assigned training environment, and identify suitable mitigations.

The five completed exercises cover SQL injection in a WHERE clause, SQL injection login bypass, file path traversal, reflected cross-site scripting, and user role control through a request parameter.

3. Environment and Tools

Operating system: Kali GNU/Linux Rolling 2026.3 Browser: Mozilla Firefox 153.4.0 ESR Testing tool: Burp Suite Community Edition 2026.9 Additional tools: Terminal, curl 8.22.0, Python 3.14.7, zip 3.0, scrot 2.0.0, Firefox Developer Tools Practice platform: PortSwigger Web Security Academy Virtualization platform: Oracle VirtualBox

Project directory: /home/kali/SHAURYA_GUPTA/Intermediate_Project_2_PortSwigger

The environment log confirms the installed operating system, Python version, Burp Suite executable, and Firefox executable. The screenshots provide evidence of the five Academy lab exercises.

4. Scope and Methodology

Scope: Only the five assigned PortSwigger Academy lab instances.

Methodology: 1. Review the relevant vulnerability learning material. 2. Explore normal application behavior. 3. Capture and inspect relevant HTTP requests and responses. 4. Perform controlled tests within the assigned lab. 5. Verify completion using the Academy's lab status. 6. Save screenshots and record observations. 7. Recommend mitigations based on the observed behavior.

All practical testing was conducted against the assigned intentionally vulnerable training labs. The assessment did not target unrelated websites.

5. Lab 1: SQL Injection in a WHERE Clause

Lab title: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Objective: Determine whether input to a database filter can alter the application's intended query behavior.

Procedure: 1. Opened the assigned SQL injection lab on PortSwigger Web Security Academy. 2. Examined the online shop and its product category filtering. 3. Observed how the selected category

affected the displayed products and the application's request. 4. Tested the relevant category input using the lab's SQL injection technique. 5. Examined the resulting product listing. 6. Verified the Academy's successful lab completion indicator.

Evidence: screenshots/lab1_sql_injection/01.png screenshots/lab1_sql_injection/02.png screenshots/lab1_sql_injection/03.png screenshots/lab1_sql_injection/04.png

Observed result: The supplied screenshots show the assigned lab and its solved confirmation. The exercise demonstrates that unsafe SQL query construction can allow a category filter to return records that were intended to remain hidden.

Security impact: An attacker may retrieve unreleased or otherwise restricted database records by manipulating an input used in a SQL query. This can expose information that the application intended to exclude from normal results.

Recommended mitigation: Use parameterized queries, avoid SQL string concatenation, apply least-privilege database permissions, and test query handling with security-focused automated tests.

6. Lab 2: SQL Injection Login Bypass

Lab title: SQL injection vulnerability allowing login bypass

Objective: Evaluate whether unsafe query construction can bypass the application's authentication logic.

Procedure: 1. Opened the assigned SQL injection login bypass lab. 2. Accessed the login page and examined the available fields. 3. Submitted a login test using the lab's intended procedure. 4. Tested the username input using the SQL injection technique specified for the exercise. 5. Examined the resulting account page. 6. Verified the Academy's successful lab completion indicator.

Evidence: screenshots/lab2_login_bypass/01.png screenshots/lab2_login_bypass/02.png screenshots/lab2_login_bypass/03.png

Observed result: The supplied screenshots show the login page, a login test, and the Academy's solved confirmation. The exercise demonstrates how an improperly constructed SQL query can undermine the application's authentication checks.

Security impact: An authentication bypass may expose protected accounts, private information, or privileged functionality.

Recommended mitigation: Use parameterized queries, securely verify passwords, implement server-side authentication, and test login logic against injection attacks.

7. Lab 3: File Path Traversal

Lab title: File path traversal, simple case

Objective: Determine whether user-controlled file paths can escape their intended directory.

Procedure: 1. Opened the assigned file path traversal lab. 2. Examined the shop's product images and associated file requests. 3. Identified a request containing a filename parameter. 4. Tested the filename parameter using the path traversal technique specified in the lab. 5. Examined the server response. 6. Verified the Academy's successful lab completion indicator.

Evidence: screenshots/lab3_path_traversal/01.png screenshots/lab3_path_traversal/02.png screenshots/lab3_path_traversal/03.png screenshots/lab3_path_traversal/04.png

Observed result: The supplied screenshots show the normal image request, the file response, and the Academy's solved confirmation. The exercise demonstrates that an insufficiently restricted file path can permit access to files outside the intended image directory.

Security impact: Sensitive files may be exposed if filesystem access is not restricted to approved resources.

Recommended mitigation: Canonicalize and validate paths, enforce directory boundaries, map identifiers to approved files, and run the application with minimum filesystem privileges.

8. Lab 4: Reflected Cross-Site Scripting

Lab title: Reflected XSS into HTML context with nothing encoded

Objective: Determine whether untrusted input is interpreted as executable browser content.

Procedure: 1. Opened the assigned reflected XSS lab. 2. Located the application's search input. 3. Submitted a normal search term and observed the results page. 4. Tested the input using the harmless JavaScript alert proof of concept specified for the exercise. 5. Observed the browser's response to the submitted input. 6. Verified the Academy's successful lab completion indicator.

Evidence: screenshots/lab4_reflected_xss/01.png screenshots/lab4_reflected_xss/02.png screenshots/lab4_reflected_xss/03.png

Observed result: The supplied screenshots show the search page, the browser alert test, and the Academy's solved confirmation. The exercise demonstrates that untrusted input reflected into an HTML context without appropriate handling can execute as browser content.

Security impact: Successful XSS may allow attacker-controlled actions in a victim's browser under the application's origin.

Recommended mitigation: Use context-aware output encoding, safe templating, safe DOM APIs, and a suitable Content Security Policy.

9. Lab 5: Broken Access Control

Lab title: User role controlled by request parameter

Objective: Test whether a client-controlled role value improperly influences authorization decisions.

Procedure: 1. Opened the assigned user role control lab. 2. Examined the application's normal pages and account access. 3. Accessed the login page and used the training credentials supplied by the Academy. 4. Inspected the lab's authorization behavior and the relevant client-controlled cookie using browser developer tools. 5. Tested the role-related value according to the lab procedure. 6. Accessed the administrative functionality. 7. Completed the required user-management action in the lab. 8. Verified the Academy's successful lab completion indicator.

Evidence: screenshots/lab5_access_control/01.png screenshots/lab5_access_control/02.png screenshots/lab5_access_control/03.png screenshots/lab5_access_control/04.png screenshots/lab5_access_control/05.png screenshots/lab5_access_control/06.png

Observed result: The supplied screenshots show the lab pages, account access, administrative functionality, and the Academy's solved confirmation. The exercise demonstrates the risk of trusting a client-controlled role value when making authorization decisions.

Security impact: Improper authorization can allow users to access data or operations beyond their assigned permissions.

Recommended mitigation: Enforce authorization server-side on every protected request. Derive roles from trusted server-side data and test for privilege escalation.

10. Results Summary

Lab	Vulnerability	Actual status
1	SQL injection in WHERE clause	Solved
2	SQL injection login bypass	Solved
3	Path traversal	Solved
4	Reflected XSS	Solved
5	Broken access control	Solved

All five assigned labs are marked as solved in the supplied screenshots.

11. General Recommendations

1. Use parameterized database queries.
2. Apply context-aware output encoding.
3. Validate and safely resolve file paths.
4. Enforce authorization on the server.
5. Follow least-privilege principles.
6. Maintain security regression tests.
7. Log relevant security events without recording unnecessary passwords, tokens, or personal data.

12. Conclusion

The project covered five common web application security vulnerabilities through practical exercises on PortSwigger Web Security Academy.

The completed labs covered SQL injection in a WHERE clause, SQL injection login bypass, file path traversal, reflected cross-site scripting, and authorization weaknesses involving a client-controlled role value.

The exercises demonstrated how unsafe database query construction can expose hidden records or undermine login checks, how insufficient file path restrictions can expose server files, how reflected untrusted input can execute as browser content, and how trusting client-controlled role values can compromise authorization.

The main defensive lessons are to use parameterized SQL queries, safely handle output and file paths, and enforce authorization using trusted server-side information.

The supplied evidence package contains screenshots for all five labs, including solved confirmations. These screenshots support the recorded completion status.

13. Evidence Index

Lab 1: SQL Injection in a WHERE Clause - screenshots/lab1_sql_injection/01.png - screenshots/lab1_sql_injection/02.png - screenshots/lab1_sql_injection/03.png - screenshots/lab1_sql_injection/04.png

Lab 2: SQL Injection Login Bypass - screenshots/lab2_login_bypass/01.png - screenshots/lab2_login_bypass/02.png - screenshots/lab2_login_bypass/03.png

Lab 3: File Path Traversal - screenshots/lab3_path_traversal/01.png - screenshots/lab3_path_traversal/02.png - screenshots/lab3_path_traversal/03.png - screenshots/lab3_path_traversal/04.png

Lab 4: Reflected Cross-Site Scripting - screenshots/lab4_reflected_xss/01.png - screenshots/lab4_reflected_xss/02.png - screenshots/lab4_reflected_xss/03.png

Lab 5: Broken Access Control - screenshots/lab5_access_control/01.png - screenshots/lab5_access_control/02.png - screenshots/lab5_access_control/03.png - screenshots/lab5_access_control/04.png - screenshots/lab5_access_control/05.png - screenshots/lab5_access_control/06.png

Environment and terminal records: - logs/environment.txt - logs/terminal-20261025-045808.log

The screenshot filenames above correspond to the supplied evidence ZIP. The environment and terminal records are listed as project artifacts and should be included in the submission directory if available.

14. References

PortSwigger Web Security Academy: https://portswigger.net/web-security

All labs: https://portswigger.net/web-security/all-labs

Burp Suite documentation: https://portswigger.net/burp/documentation